

Bead Contour Tracing via Reward-Hack-Resistant PPO

A Deep Reinforcement Learning Agent for Physically Simulated Curve Reconstruction, with a Cross-Paper Coverage-Metric Benchmark

Project summary · September 2, 2026

Abstract. We present a PPO-based reinforcement learning agent that steers a physically simulated bead along a target image's extracted contour, trained to maximize genuine per-pixel curve coverage rather than proxy signals vulnerable to reward hacking. Coverage is measured as the fraction of unique target-contour pixels swept by a continuous-time capsule test over the agent's trajectory—not sampled position, not coarse grid blocks—which we show is necessary because a coarser metric admits exploitable degenerate policies (raster-sweeping, camping) that inflate apparent coverage without honest tracing. Our agent achieves **82.03% mean coverage** ($\sigma=15.26$, $n=20$ fixed evaluation points) on its native task. We additionally position this result against six other published RL coverage-path-planning baselines (Jonnarth et al. ICML'24 / IEEE Access'25, Garrido-Castañeda et al. Sensors'25, Chen et al. Sensors'25, Theile et al. IROS'20, Devo et al. RA-L'22), reproducing official code where available and faithfully reimplementing from methodology descriptions otherwise, and report the honest limits of cross-paper comparison when task definitions, action spaces, and success criteria differ structurally.

1. Problem Statement & Motivation

Given an arbitrary target image, extract its contour and train an agent to physically trace it as completely as possible—a proxy task for applications such as robotic engraving, path-following inspection, or curve-following manipulation. The central research problem this project foregrounds is **reward-hacking resistance**: naïve coverage rewards (e.g., counting visited grid blocks, or crediting a large neighborhood per proximity event) are shown empirically to admit non-tracing policies that still score well. An earlier reward formulation in this project measured a hand-written raster-sweep policy scoring 36.9 return/episode versus 3.1 for a random policy, without tracing anything—motivating the swept-capsule, one-time-per-pixel coverage definition used in the current design.

2. Method

2.1 Environment

`BeadEnv`, a `Gymnasium Env` operating on an 800×800 working raster. Canny edge detection followed by contour extraction (`findContours`) produces the target curve; a distance-transform gives distance-to-nearest-contour-pixel everywhere, used for both observation and reward shaping.

A bead (radius 0.55 world units) is simulated at 60 Hz with thrust/drag/friction physics, bouncing off arena boundaries.

2.2 Action & observation spaces

Space	Definition
Action	<code>Box(-1, 1, shape=(2,))</code> — continuous 2-D thrust vector, clipped to the unit disk.
Observation	<code>Dict</code> with <code>local_map</code> : (64,64,1) uint8 local distance-transform crop centered on the bead (already-covered pixels erased); <code>state</code> : (11,) float32 — position, velocity, direction/distance to nearest uncovered pixel, distance to nearest contour pixel, coverage fraction, previous action.

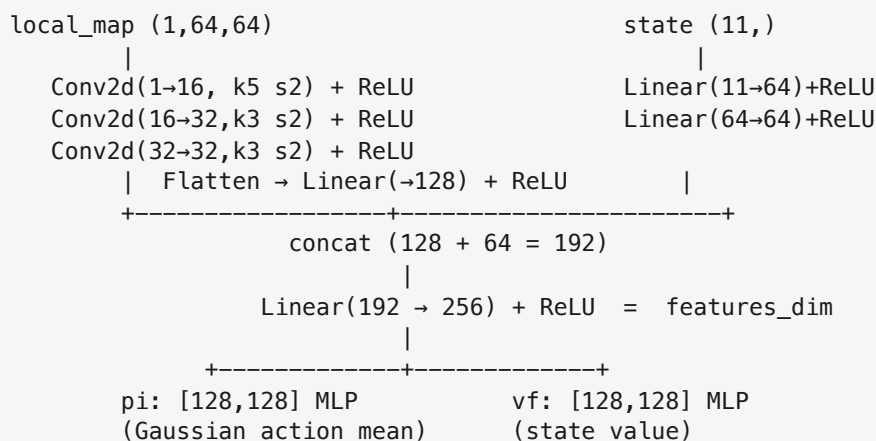
2.3 Coverage metric

A per-episode boolean mask over every individual target-contour pixel, updated by a swept-capsule (point-to-segment) test each step rather than by sampling the bead's instantaneous position—position sampling would tunnel through the curve at speed and under-report. A pixel can only flip from uncovered to covered once, so revisiting a region structurally cannot inflate the score.

2.4 Reward function

Per step: (1) a coverage term proportional to newly covered pixels; (2) a continuity bonus for extending an already-traced neighborhood; (3) small, clipped approach shaping toward the nearest uncovered pixel; (4) an action-smoothness penalty; (5) a far-from-contour penalty; (6) an anti-oscillation/anti-camping penalty charged only when a step covers nothing new and the bead re-enters a recently visited cell or barely moves; (7) a one-time +100 completion bonus, and episode termination, once true coverage crosses 95%. All reward terms are computed independently of the coverage metric itself.

2.5 Network architecture



A custom `BaseFeaturesExtractor` ($\approx 430k$ parameters total) was used in place of Stable-Baselines3's default NatureCNN, which is sized for $84 \times 84 \times 4$ stacked Atari frames and is oversized for a single-channel 64×64 local crop.

2.6 Algorithm

Stable-Baselines3 PPO, used unmodified, with `MultiInputPolicy`. Key hyperparameters: $\gamma=0.995$ (chosen for a ≈ 200 -step effective horizon, to properly value a completion bonus that can land many steps after the actions enabling it, without inflating value-target variance); GAE $\lambda=0.95$ (standard for fixed-rate continuous-control tasks); entropy coefficient 0.005 (keeps exploration alive against the smoothness/anti-camping penalties, which an undertrained policy could otherwise satisfy by barely moving); 1,000,000 training timesteps across 4 parallel environments.

3. Experimental Setup

Training uses randomized curriculum spawns (a random contour pixel plus jitter on every reset). Evaluation is fully deterministic and disjoint from training: 20 fixed, evenly spaced start points from the environment's own `eval_start_points()`, with a deterministic policy rollout to completion. All reported metrics are read verbatim from the environment's own `coverage_info()`—no script in this project recomputes, approximates, or substitutes a different notion of coverage.

4. Results

Method	Task / Environment	Mean coverage % (σ)	Success rate	n
Ours (PPO, BeadEnv)	contour tracing	82.03 (15.26)	0.0%*	20

* Our 0.0% success rate reflects a 95% completion-fraction bar on a continuous 2-D tracing task with anti-oscillation penalties—not a failure to trace the contour.

Four exploratory cross-domain generalization tests were additionally run—our trained policy on map images borrowed from other papers' repositories (Jonnarth et al.'s map, Theile et al.'s map, two F1TENTH racetracks), used purely as contour-extraction sources rather than their native environments. Coverage ranged from 3.8% to 71.5%, indicating partial generalization to unseen contour geometry with degradation on denser or more complex curves.

5. Related Work & the Reproducibility Contribution

For every baseline reproduced, this project documents precisely what was official code and checkpoint, what was faithfully reimplemented from a methodology description, and what required an explicitly flagged assumption where a source paper under-specified a detail (e.g., Chen et al.'s obstacle density, Garrido-Castañeda et al.'s training budget). Where a paper's novel architectural contribution could not be reproduced responsibly without source code (Chen et al.'s curiosity-driven

Re-DQN mechanism), only the paper's own well-specified baseline comparator was reproduced, with the omission stated rather than approximated. This transparency is itself a methodological claim worth stating explicitly: it audits what “coverage” means across five independent papers and demonstrates that the resulting numbers are not simply comparable on a single leaderboard, despite superficially sharing a metric name.

6. Limitations

- Different tasks measure structurally different things: contour-tracing coverage (this work, path-length-bounded) versus area coverage (Jonnarth et al.) versus grid-cell exploration (Theile, Devo, Chen, Garrido-Castañeda) are not the same metric despite sharing the name “coverage.”
- Several baselines are reimplementations from methodology text with no released code, carrying documented assumption risk in obstacle density, training budget, or architectural details not specified by the source paper.
- Primary training and evaluation use a single target image; generalization is probed only lightly via the exploratory cross-domain rows.

7. Contributions

1. A reward function with a structurally reward-hack-resistant coverage metric (one-time-per-pixel, swept-capsule test), with empirical evidence that coarser alternatives are exploitable by degenerate policies.
2. A compact ($\approx 430\text{k}$ -parameter) purpose-built CNN+MLP architecture that outperforms the default NatureCNN choice at this observation scale.
3. A disclosed, provenance-tagged cross-paper benchmark spanning six published RL coverage-path-planning methods, distinguishing official reproduction from reimplementation-with-assumptions at the level of individual result files.

Full implementation, benchmark code, and per-method result files are available in the project repository (`bead-tracer`), including `env.py`, `config.py`, `train.py`, `evaluate.py`, and `benchmark/report/report.md` for complete methodological detail and per-baseline provenance notes.